Universidad Rafael Landívar.
Facultad de Ingeniería.
Licenciatura en Ingeniería en Informática y Sistemas.
Lenguajes Formales y Autómatas
Docente: Ing. Jorge Estuardo Tello Monzón

INFORME PROYECTO FINAL

“Sistema De Rutas Aéreas Y Autómatas Ponderados (Aeroroute) Modelado Formal, Algoritmo De Ruta Óptima Y Validación Estocástica”

Estudiante:
Luis Manuel Velásquez González, 1502325
Fernando Méndez, 1601524
Morelia González Ixcot, 1536325
Josue Sebastián Soch Chaclan, 1537924
Manuel Alejandro Coxaj Ramírez, 1506925

Quetzaltenango, 03 de noviembre 2026

1. INTRODUCCIÓN

El presente informe técnico documenta el diseño, fundamentación matemática, arquitectura de software y validación experimental del sistema AeroRoute (Conexiones de Vuelo). El proyecto aborda el problema de la planificación y validación de rutas óptimas dentro de una red de transporte aéreo internacional, integrando los principios de la Teoría de Lenguajes Formales y Autómatas con algoritmos de optimización en grafos dirigidos ponderados y modelos estocásticos de predicción.

En la ingeniería de sistemas de transporte aéreo, una red de conexiones entre aeropuertos no solo constituye una estructura topológica de datos, sino un sistema formal de transiciones de estado donde cada itinerario válido representa una palabra aceptada por una máquina abstracta. Bajo este enfoque, AeroRoute modela la red aeroportuaria mediante un Autómata Finito Determinista Ponderado (WFA), emplea el algoritmo de Dijkstra con colas de prioridad binarias para la determinación del camino de costo mínimo y complementa el análisis operativo mediante Cadenas de Markov para estimar la probabilidad de estados de vuelo (A tiempo, Retrasado, Cancelado) a lo largo de múltiples períodos.

2. ARQUITECTURA DEL SISTEMA Y ESPECIFICACIÓN TÉCNICA

2.1. Stack Tecnológico y Diseño Libre de Dependencias

El sistema fue diseñado bajo una arquitectura cliente-servidor modular, priorizando la pureza algorítmica y el control de estructuras de datos en memoria:

- **Backend (Python 3.9+):** Desarrollado bajo el principio de cero dependencias externas (Zero External Dependencies), utilizando exclusivamente módulos nativos de la biblioteca estándar:
    - **http.server (ThreadingHTTPServer, BaseHTTPRequestHandler):** Servidor HTTP concurrente multihilo para despachar tanto archivos estáticos como los endpoints de la API REST (/api/red y /api/ruta).
    - **heapq:** Implementación de montículo binario (min-heap) para la cola de prioridad del algoritmo de búsqueda.
    - **json y urllib.parse:** Serialización de estructuras de datos y análisis sintáctico de parámetros de consulta (query strings).
    - **typing y math:** Tipado estático formal (Dict, List, Tuple, Optional, Union) y validación de cotas numéricas finitas (isfinite).
- **Frontend (Web Estándar y SIG):**
    - **HTML5, CSS3 y JavaScript Vanilla (ES6+):** Construido sin frameworks externos, aplicando diseño responsivo mediante CSS Grid y Flexbox.
    - **Motor Cartográfico Principal (Leaflet.js v1.9.4):** Renderizado de sistemas de información geográfica (SIG) con capas de teselas CartoDB, marcadores dinámicos (L.divIcon) y trazado vectorial de aristas (L.polyline).
    - **Motor Gráfico de Contingencia (HTML5 2D Canvas):** Sistema de respaldo local autónomo (<canvas id="mapa-local">) que proyecta coordenadas geográficas, masas continentales vectoriales y animaciones de vuelo en caso de ausencia de conectividad a la red.

2.2. Organización Modular del Proyecto

La estructura del proyecto separa estrictamente la definición formal del autómata, la lógica de optimización, el modelo probabilístico y la capa de presentación:

- **aeropuertos.py:** Define la clase GrafoAeropuertos, encapsulando el registro de nodos y aristas con validaciones sintácticas de códigos IATA (3 caracteres alfabéticos ASCII en mayúsculas), prohibición de lazos propios (origen ≠ destino), control de duplicados y verificación de pesos reales estrictamente positivos (w > 0).
- **automata.py:** Contiene la definición formal del alfabeto (ALFABETO), la topología dirigida base de la red (grafo) y las funciones reconocedoras validar_ruta y calcular_peso_ruta.
- **dijkstra.py:** Implementa el motor de optimización calcular_dijkstra junto con herramientas de normalización de adyacencia y pruebas en consola.
- **markov.py:** Implementa álgebra matricial nativa en Python para computar la evolución de estados operativos de vuelo en n períodos.
- **main.py:** Actúa como orquestador central; inicializa la red con coordenadas geográficas (AEROPUERTOS), levanta el servidor en 127.0.0.1:8000 y coordina el flujo secuencial entre Dijkstra, el autómata validador y el predictor de Markov.
- **web/ (index.html, estilos.css, app.js):** Interfaz gráfica interactiva, consumo asíncrono de API y motor de animación trigonométrica de aeronaves.

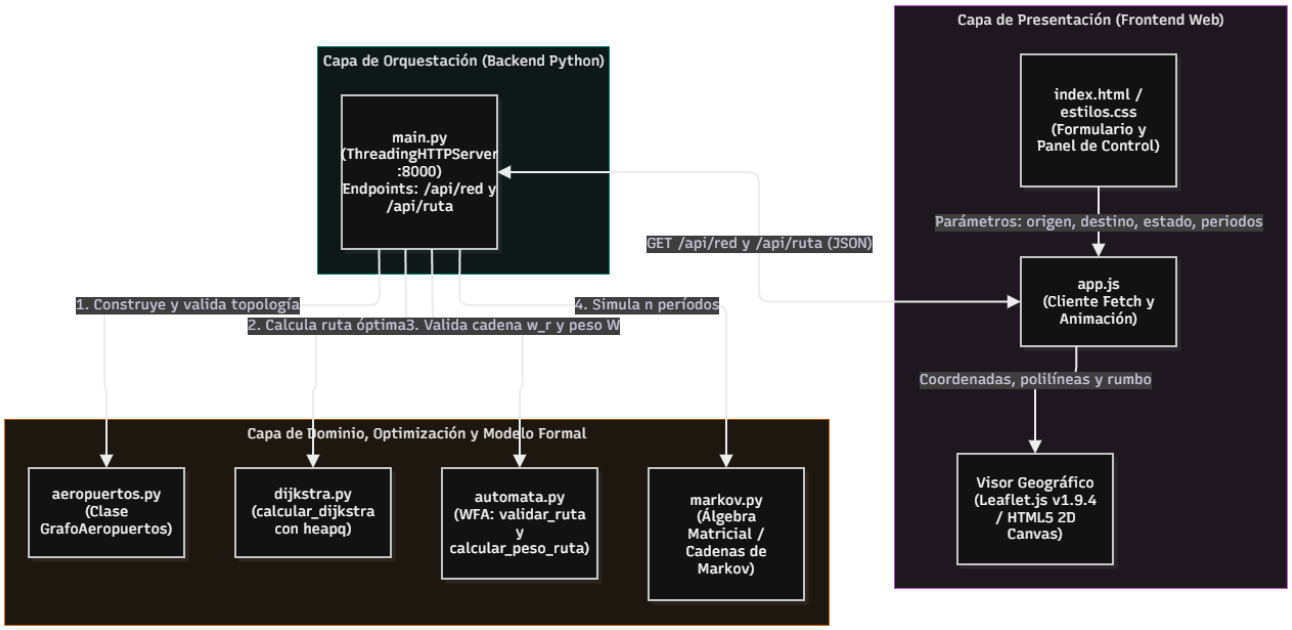


Diagrama de Arquitectura de Bloques y Flujo de Datos entre Módulos

2.3. Estructuración del Grafo en Memoria

La red aérea se modela como un Grafo Dirigido Ponderado (Dígrafo con pesos), donde cada conexión posee una dirección explícita u → v y un costo asociado w(u, v) > 0. En memoria principal, el grafo se representa mediante una Lista de Adyacencia implementada con Diccionarios Anidados (tablas hash encadenadas):

`grafo : Dict[str, Dict[str, float]]`Donde la clave de primer nivel corresponde al código IATA del nodo de salida (ej. "GUA"), la clave de segundo nivel identifica al nodo vecino de llegada (ej. "SAL") y el valor flotante almacena la distancia geográfica en kilómetros (ej. 200.0).

3. FUNDAMENTO FORMAL (TEORÍA DE AUTÓMATAS Y LENGUAJES FORMALES)

De acuerdo con los fundamentos de la teoría de la computación, la red de caminos aéreos de AeroRoute se modela formalmente como un Autómata Finito Determinista Ponderado (WFA, Weighted Finite Automaton) definido mediante la séxtupla:

$A = (Q, \Sigma, \delta, q_0, F, w)$A continuación, se define de manera exhaustiva cada componente formal y su correspondencia exacta con la implementación computacional del sistema:

3.1. Alfabeto (Σ)

El alfabeto Σ es el conjunto finito y no vacío de símbolos terminales que representan los identificadores de los nodos hacia los cuales es posible efectuar una transición (aristas o tramos de vuelo dentro de la red). En el sistema, está definido explícitamente en la estructura ALFABETO del módulo automata.py:

`Σ = { GUA, SAL, MEX, MIA, JFK, LAX, MAD, PTY, CHI, BR }`

3.2. Lenguaje Formal (L)

El lenguaje $L(A) \subseteq \Sigma^+$ reconocido por el autómata es el conjunto de todas las cadenas finitas de símbolos $w_r = q_0\ q_1\ q_2\ ...\ q_k$ (con $k \geq 1$) que representan caminos dirigidos válidos, continuos y conexos dentro del grafo desde el aeropuerto de origen hasta el aeropuerto de destino.

Formalmente, una cadena $w_r = q_0\ q_1\ ...\ q_k$ pertenece al lenguaje L(A) si y solo si cumple simultáneamente con las siguientes condiciones:

1. **Condición de inicio:** El primer símbolo de la cadena corresponde al estado inicial definido para la consulta (q_0 = origen).
2. **Condición de aceptación:** El último símbolo de la cadena pertenece al conjunto de estados finales ($q_k \in F$, donde F = {destino}).
3. **Pertenencia léxica:** Para todo índice $i \in \{0, 1, ..., k\}$, se cumple que $q_i \in \Sigma$ y $q_i \in Q$.

4. **Continuidad transicional:** Para todo paso intermedio i ∈ {0, 1, ..., k - 1}, existe una transición válida definida en la función δ tal que δ(q_i, q_{i+1}) = q_{i+1}.

3.3. Definición del Autómata (A)

- **Conjunto de Estados (Q):** Representa el conjunto finito de vértices o nodos del dígrafo (los 10 aeropuertos registrados en la topología del sistema, cada uno con metadatos geográficos de nombre, latitud y longitud):
  `Q = { GUA (Ciudad de Guatemala), SAL (San Salvador), MEX (Ciudad de México), MIA (Miami), JFK (Nueva York), LAX (Los Ángeles), MAD (Madrid), PTY (Ciudad de Panamá), CHI (Chicago), BR (Brasilia) }`
- **Función de Transición (δ):** Es una función parcial determinista δ : Q × Σ → Q que define las aristas dirigidas válidas entre nodos, donde δ(u, v) = v sí y solo si existe un vuelo directo registrado desde el aeropuerto u hacia el aeropuerto v. Si no existe conexión directa, δ(u, v) = ∅ (indefinida), provocando el rechazo inmediato de la cadena.
  Las 18 transiciones dirigidas activas definidas en el sistema son:
  - Desde GUA: δ(GUA, SAL) = SAL, δ(GUA, MEX) = MEX, δ(GUA, PTY) = PTY
  - Desde SAL: δ(SAL, MEX) = MEX, δ(SAL, MIA) = MIA
  - Desde MEX: δ(MEX, MIA) = MIA, δ(MEX, LAX) = LAX
  - Desde MIA: δ(MIA, JFK) = JFK, δ(MIA, MAD) = MAD
  - Desde JFK: δ(JFK, MAD) = MAD
  - Desde LAX: δ(LAX, JFK) = JFK
  - Desde PTY: δ(PTY, MIA) = MIA, δ(PTY, MAD) = MAD, δ(PTY, CHI) = CHI
  - Desde CHI: δ(CHI, GUA) = GUA, δ(CHI, BR) = BR
  - Desde BR: δ(BR, MIA) = MIA, δ(BR, MEX) = MEX
  - Desde MAD: ∅ (Estado pozo o sumidero; no posee transiciones de salida).
- **Estado Inicial ($q_0 \in Q$):** Representa el nodo origen de la consulta seleccionada por el usuario (parámetro origen).
- **Conjunto de Estados de Aceptación ($F \subseteq Q$):** Conjunto unitario F = { destino } que representa el nodo meta de la consulta seleccionada por el usuario (parámetro destino, con la restricción operativa $q_0 \neq$ destino).
- **Función de Peso (w):** Es una función de costo escalar positiva w : Q × Σ → $\mathbb{R}^+$ que asigna a cada transición válida δ(u, v) su distancia geográfica en kilómetros. Opera algebraicamente sobre el semianillo tropical ($\mathbb{R}^+ \cup \{\infty\}$, min, +), donde el peso acumulado de una cadena válida w_r = q_0 q_1 ... q_k se define como la suma de los pesos de sus transiciones individuales:
  `𝕎(w_r) = Σ (desde i = 0 hasta k - 1) w(q_i, q_{i+1})`

3.4. Tabla de Correspondencia Teórico-Computacional

Componente Formal	Definición Matemática	Implementación en Código AeroRoute
Estados (Q)	Conjunto finito de 10 vértices aeroportuarios.	Claves del diccionario grafo en automata.py, atributo self.grafo en aeropuertos.py y catálogo AEROPUERTOS en main.py.
Alfabeto (Σ)	Conjunto de símbolos terminales de transición.	Lista explícita ALFABETO en automata.py.
Transiciones (δ)	Función parcial $\delta : Q \times \Sigma \rightarrow Q$.	Diccionario de adyacencia grafo[actual]. Evaluada en validar_ruta mediante: if siguiente not in grafo[actual]: return False.
Estado Inicial (q_0)	Nodo origen del itinerario ($q_0 \in Q$).	Parámetro origen validado en validar_ruta mediante: if ruta[0] != origen: return False.
Estados de Aceptación (F)	Subconjunto destino $F = \{destino\} \subseteq Q$.	Parámetro destino validado en validar_ruta mediante: if ruta[-1] != destino: return False.
Función de Peso (w)	Aplicación $w : Q \times \Sigma \rightarrow \mathbb{R}^+$ (distancia en km).	Valor flotante en grafo[actual][siguiente] sumado por calcular_peso_ruta y contrastado con Dijkstra.

Diagrama de Transición de Estados del Autómata Ponderado $A = (Q, \Sigma, \delta, q_0, F, w)$ con los 10 nodos y las 18 aristas dirigidas etiquetadas con sus pesos en kilómetros]
Diagrama a detalle en la carpeta "Diagramas y Documentos" del repositorio.

3.5. Doble Verificación y Reconocimiento Formal en el Backend

En AeroRoute, el cálculo de ruta óptima y el reconocimiento formal operan de manera acoplada pero independiente para garantizar integridad matemática. Una vez que el algoritmo de optimización propone una secuencia de nodos, el orquestador (main.py) somete la secuencia a la función validar_ruta(recorrido, origen, destino, RED.grafo) en automata.py y seguidamente ejecuta una comprobación cruzada de pesos mediante calcular_peso_ruta, verificando que $|W(w_r) - distancia_dijkstra| \leq 10^{-6}$ antes de emitir cualquier respuesta al cliente.

4. ALGORITMO UTILIZADO Y ESTRUCTURAS DE DATOS

4.1. Selección y Justificación Algorítmica

Para resolver el problema del camino de costo mínimo desde un nodo origen q_0 hasta un estado de aceptación en F dentro de un dígrafo con pesos estrictamente positivos (w(u, v) >

0 ∀ (u, v) ∈ δ), se implementó el Algoritmo de Dijkstra en el módulo dijkstra.py bajo la función calcular_dijkstra(grafo, origen, destino).

La elección de Dijkstra con cola de prioridad binaria garantiza la obtención de la cadena óptima w_r* ∈ L(A) tal que:

$$W(w_r^*) = \min \{ W(w_r) \mid w_r \in L(A) \}$$

4.2. Descripción Paso a Paso del Funcionamiento

El flujo algorítmico implementado en calcular_dijkstra consta de cuatro fases rigurosas:

1. **Normalización y Validación de Precondiciones:**
   Se limpian y transforman a mayúsculas las cadenas origen y destino (.strip().upper()). Si alguno de los nodos no pertenece al conjunto Q del grafo, la función retorna (None, [], []). Si origen == destino, retorna inmediatamente (0.0, [origen], []).
2. **Inicialización de Estructuras Auxiliares:**
   - Diccionario de distancias mínimas (distancias: Dict[str, float]): Se inicializa asignando infinito positivo float("inf") a cada estado q ∈ Q, excepto al estado inicial: distancias[origen] = 0.0.
   - Diccionario de predecesores (predecesores: Dict[str, Optional[str]]): Punteros de retroceso inicializados en None para reconstruir el camino óptimo.
   - Cola de prioridad (cola_prioridad: List[Tuple[float, str]]): Montículo binario mínimo gestionado con heapq, inicializado con la tupla [(0.0, origen)].
   - Conjunto de estados cerrados (visitados: set): Estructura hash para registrar los nodos cuya distancia mínima definitiva ya ha sido fijada.
3. **Ciclo Principal de Exploración y Relajación de Aristas:**
   Mientras la cola de prioridad no esté vacía:
   - Se extrae la tupla (distancia_actual, actual) con menor costo acumulado mediante heapq.heappop(cola_prioridad).
   - Eliminación perezosa (Lazy Deletion): Si el nodo actual ya pertenece al conjunto visitados, la iteración se descarta con continue. En caso contrario, se añade a visitados.
   - Parada Temprana (Early Termination): Si actual == destino, el ciclo se interrumpe con break, dado que por la propiedad voraz de Dijkstra sobre pesos positivos, la distancia extraída de la cima del montículo hacia el destino ya es globalmente mínima.
   - Relajación (Relaxation): Para cada par (vecino, peso) obtenido mediante _obtener_vecinos(grafo, actual), si el vecino no ha sido visitado, se calcula nueva_distancia = distancia_actual + peso. Si nueva_distancia < distancias[vecino], se actualiza distancias[vecino] = nueva_distancia, se asigna predecesores[vecino] = actual y se inserta (nueva_distancia, vecino) en el montículo con heapq.heappush.
4. **Verificación de Conectividad y Reconstrucción del Camino:**
   Al concluir el bucle, si distancias[destino] == float("inf"), significa que no existe ninguna secuencia de transiciones válida en L(A) entre ambos nodos, retornando (None, [], []). Si es alcanzable, se reconstruye la lista recorrido siguiendo los

punteros predecesores desde destino hasta origen, invirtiendo el resultado con recorrido.reverse(). Finalmente, las escalas intermedias se obtienen mediante particionado de listas: escalas = recorrido[1:-1].

4.3. Complejidad Computacional

Sea |V| el número de estados (|Q| = 10) y |E| el número de transiciones dirigidas (|δ| = 18):

- **Complejidad Temporal:** O((|V| + |E|) log |V|), derivada de a lo sumo |V| extracciones útiles del montículo binario y a lo sumo |E| inserciones por relajación de aristas, cada una con costo logarítmico en el tamaño del montículo.
- **Complejidad Espacial:** O(|V|), requerida para almacenar en memoria los diccionarios distancias, predecesores, el conjunto visitados y el montículo de prioridad.

4.4. Implementación del Algoritmo en Código Fuente (dijkstra.py)

```
import heapq
from typing import Dict, List, Tuple, Optional

def _obtener_vecinos(grafo, nodo: str):
    if hasattr(grafo, "obtener_destinos"):
        return grafo.obtener_destinos(nodo).items()
    if isinstance(grafo, dict) and nodo in grafo:
        return grafo[nodo].items()
    return [].items() if hasattr(grafo, "obtener_destinos") else {}

def calcular_dijkstra(grafo, origen: str, destino: str) -> Tuple[Optional[float],
List[str], List[str]]:
    origen = str(origen).strip().upper()
    destino = str(destino).strip().upper()

    todos_nodos = set(grafo.obtener_aeropuertos() if hasattr(grafo,
"obtener_aeropuertos") else grafo.keys())
    if origen not in todos_nodos or destino not in todos_nodos:
        return None, [], []

    if origen == destino:
        return 0.0, [origen], []

    distancias: Dict[str, float] = {nodo: float("inf") for nodo in todos_nodos}
    predecesores: Dict[str, Optional[str]] = {nodo: None for nodo in todos_nodos}
    distancias[origen] = 0.0

    cola_prioridad: List[Tuple[float, str]] = [(0.0, origen)]
    visitados = set()

    while cola_prioridad:
        distancia_actual, actual = heapq.heappop(cola_prioridad)
```

```
        if actual in visitados:
            continue
        visitados.add(actual)

        if actual == destino:
            break

        vecinos = _obtener_vecinos(grafo, actual)
        for vecino, peso in (vecinos.items() if isinstance(vecinos, dict) else
vecinos):
            if vecino not in visitados:
                nueva_distancia = distancia_actual + float(peso)
                if nueva_distancia < distancias[vecino]:
                    distancias[vecino] = nueva_distancia
                    predecesores[vecino] = actual
                    heapq.heappush(cola_prioridad, (nueva_distancia, vecino))

    if distancias[destino] == float("inf"):
        return None, [], []

    recorrido = []
    nodo_actual: Optional[str] = destino
    while nodo_actual is not None:
        recorrido.append(nodo_actual)
        nodo_actual = predecesores[nodo_actual]
    recorrido.reverse()

    escalas = recorrido[1:-1]
    return distancias[destino], recorrido, escalas
```

Diagrama de Flujo del Algoritmo calcular_dijkstra con Cola de Prioridad Binaria y Reconstrucción de Predecesores] **en carpeta “Diagramas y Documentos”** para mejor visibilidad

4.5. Modelo Complementario: Predicción Estocástica con Cadenas de Markov (markov.py)

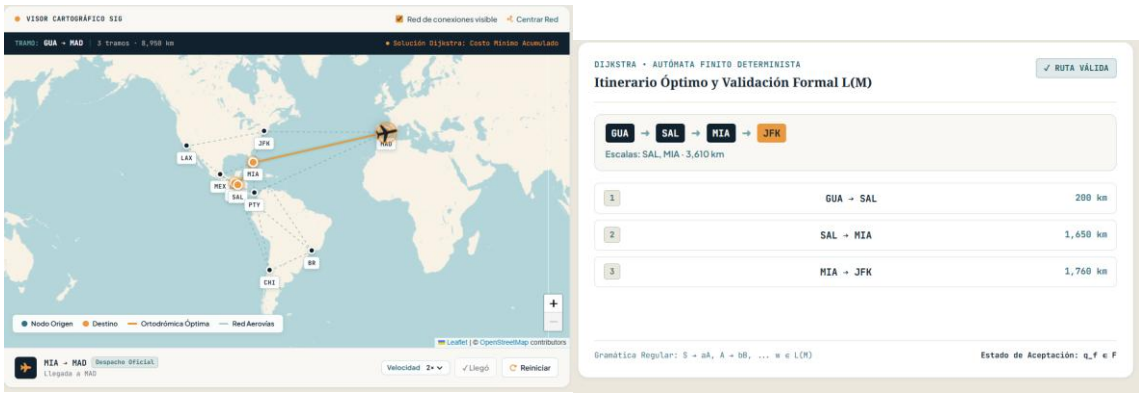
De forma complementaria a la optimización espacial, el sistema incorpora un modelo estocástico discreto basado en Cadenas de Markov sobre el espacio de estados operativos S = {"A tiempo", "Retrasado", "Cancelado"}. A partir de un vector de estado inicial v_0 y una matriz de transición estocástica P de dimensión 3×3 implementada con álgebra matricial pura en Python, el sistema calcula la distribución de probabilidad tras n períodos ($v_n = v_0 \cdot P^n$), entregando tanto el historial período a período como el estado operativo más probable al arribar al destino.

5. EJEMPLOS DE EJECUCIÓN Y CASOS DE PRUEBA

Para validar el cumplimiento de los requerimientos funcionales, la exactitud del algoritmo de Dijkstra y el reconocimiento formal del autómata, se documentan cuatro casos de prueba ejecutados sobre la topología oficial del sistema (10 nodos y 18 aristas dirigidas).

5.1. Caso de Prueba 1: Ruta Transatlántica Multiescala (GUA → MAD)

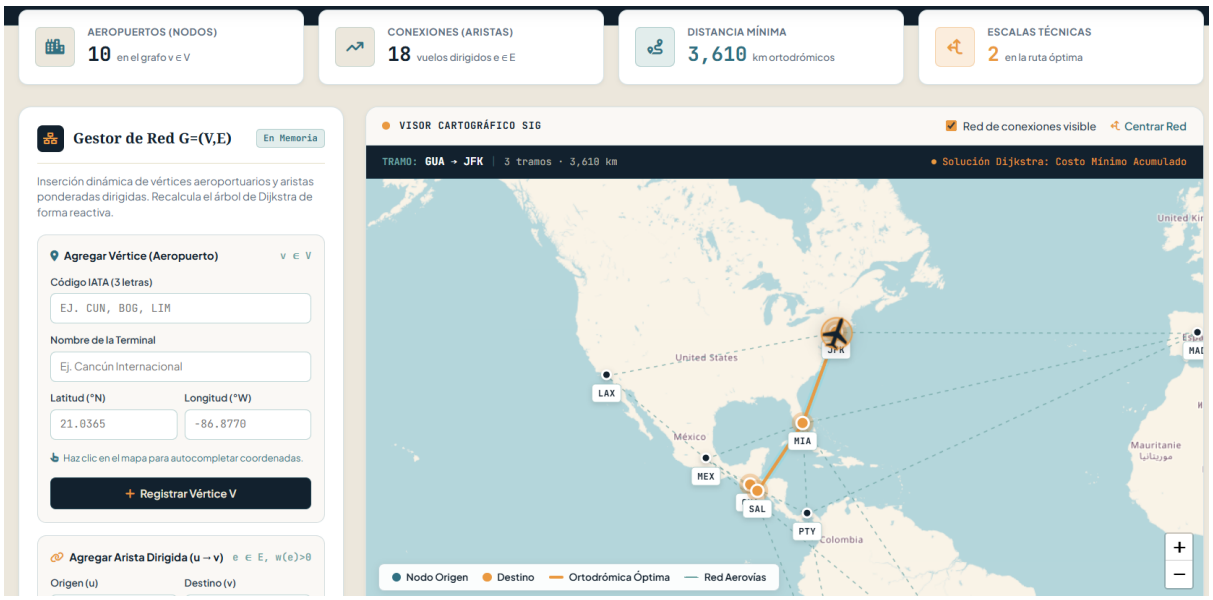
- **Parámetros de Entrada:**
  - Estado Inicial (q_0): "GUA" (Ciudad de Guatemala)
  - Estado de Aceptación (F): {"MAD"} (Madrid, España)
  - Estado Inicial de Markov: "A tiempo" | Períodos (n): 3
- **Evaluación de Caminos Candidatos en L(A):**
  El grafo presenta múltiples cadenas válidas entre GUA y MAD:
  a. w_1 = GUA → SAL → MIA → MAD: $W(w_1)$ = 200 + 1650 + 7100 = 8,950.00 km (Mínimo Global)
  b. w_2 = GUA → SAL → MIA → JFK → MAD: $W(w_2)$ = 200 + 1650 + 1760 + 5760 = 9,370.00 km
  c. w_3 = GUA → PTY → MAD: $W(w_3)$ = 1350 + 8200 = 9,550.00 km
  d. w_4 = GUA → MEX → MIA → MAD: $W(w_4)$ = 1060 + 2060 + 7100 = 10,220.00 km
  e. w_5 = GUA → PTY → MIA → MAD: $W(w_5)$ = 1350 + 1860 + 7100 = 10,310.00 km
- **Resultados Obtenidos:**
  - Cadena Óptima Seleccionada: ["GUA", "SAL", "MIA", "MAD"]
  - Escalas Intermedias (2): ["SAL", "MIA"]
  - Desglose de Transiciones: δ(GUA, SAL) = 200.00 km; δ(SAL, MIA) = 1,650.00 km; δ(MIA, MAD) = 7,100.00 km.
  - Costo Total (W): 8,950.00 km
  - Validación del Autómata: Valida = True ("La ruta es valida."). Diferencia de peso cruzado: 0.0 km.
  - Predicción de Markov (n = 3): Período 1 (A tiempo: 75.00%, Retrasado: 20.00%, Cancelado: 5.00%); Período 2 (A tiempo: 64.00%, Retrasado: 26.50%, Cancelado: 9.50%); Período 3 (A tiempo: 58.70%, Retrasado: 28.90%, Cancelado: 12.40%). Estado más probable: "A tiempo" (58.70%).



Interfaz Web de AeroRoute mostrando la ejecución del Caso 1 (GUA → MAD), polilínea resaltada en el mapa, desglose de 8,950 km, 2 escalas y predicción de Markov]

5.2. Caso de Prueba 2: Ruta Continental Norteamérica Este (GUA → JFK)

- **Parámetros de Entrada:**
  - Estado Inicial (q_0): "GUA" (Ciudad de Guatemala)
  - Estado de Aceptación (F): {"JFK"} (Nueva York, EE. UU.)
- **Evaluación de Caminos Candidatos en L(A):**
  a. w_1 = GUA → SAL → MIA → JFK: $W(w_1)$ = 200 + 1650 + 1760 = 3,610.00 km (Mínimo Global)
  b. w_2 = GUA → PTY → MIA → JFK: $W(w_2)$ = 1350 + 1860 + 1760 = 4,970.00 km
  c. w_3 = GUA → MEX → LAX → JFK: $W(w_3)$ = 1060 + 2500 + 3980 = 7,540.00 km
- **Resultados Obtenidos:**
  - Cadena Óptima Seleccionada: ["GUA", "SAL", "MIA", "JFK"]
  - Escalas Intermedias (2): ["SAL", "MIA"]
  - Desglose de Transiciones: δ(GUA, SAL) = 200.00 km; δ(SAL, MIA) = 1,650.00 km; δ(MIA, JFK) = 1,760.00 km.
  - Costo Total (W): 3,610.00 km
  - Validación del Autómata: Valida = True ("La ruta es valida.").

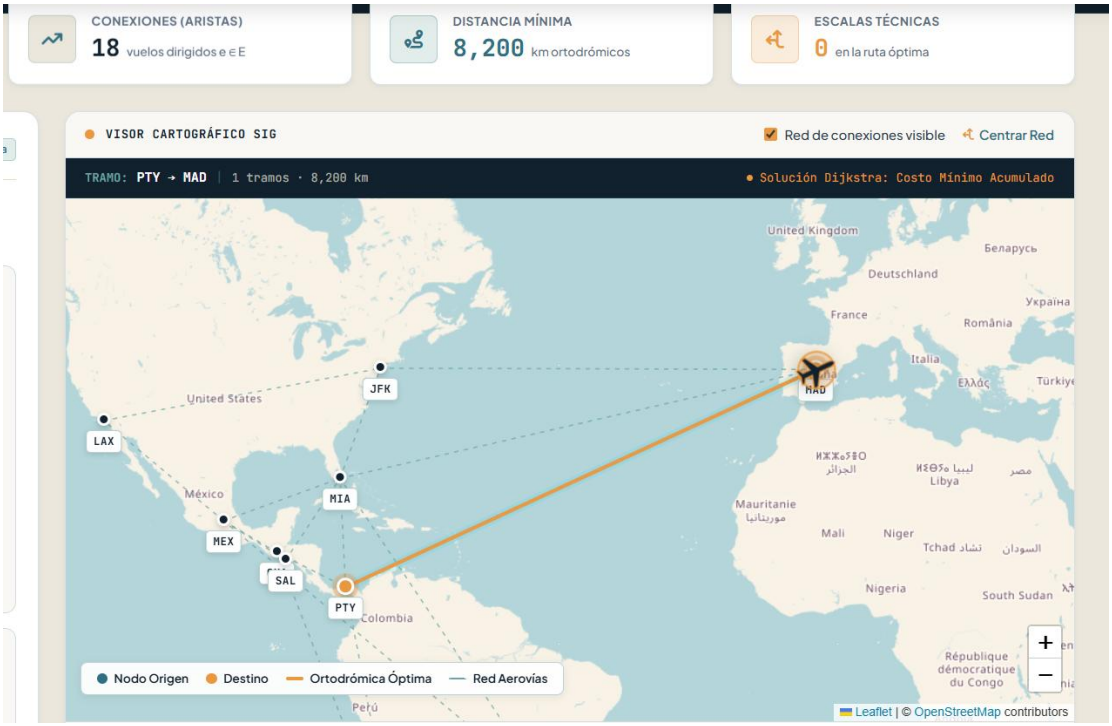


Ejecución del Caso 2 (GUA → JFK) en la interfaz gráfica

5.3. Caso de Prueba 3: Optimización de Vuelo Directo frente a Escala (PTY → MAD)

- **Parámetros de Entrada:**
  - Estado Inicial (q_0): "PTY" (Ciudad de Panamá)
  - Estado de Aceptación (F): {"MAD"} (Madrid, España)
- **Evaluación Comparativa:**
  a. w_1 = PTY → MAD (Directo): $W(w_1)$ = 8,200.00 km (Mínimo Global)
  b. w_2 = PTY → MIA → MAD (1 escala): $W(w_2)$ = 1860 + 7100 = 8,960.00 km
- **Resultados Obtenidos:**
  - Cadena Óptima Seleccionada: ["PTY", "MAD"]

- Escalas Intermedias (0): [] (Vuelo directo)
- Costo Total (W): 8,200.00 km
- Validación del Autómata: Valida = True ("La ruta es valida.").



Ejecución del Caso 3 (PTY → MAD) evidenciando la selección de transición directa con 0 escalas y 8,200 km]

5.4. Caso de Prueba 4: Prueba Negativa de Inalcanzabilidad en Dígrafo (MAD → GUA)

- **Parámetros de Entrada:**
  - Estado Inicial (q_0): "MAD" (Madrid, España)
  - Estado de Aceptación (F): {"GUA"} (Ciudad de Guatemala)
- **Análisis Formal y Traza de Ejecución:**
  En la definición de la función de transición δ, el nodo MAD es un estado sumidero (grafo["MAD"] = {}), por lo que no existen transiciones salientes δ(MAD, σ) para ningún símbolo σ ∈ Σ. En consecuencia, el lenguaje reconocido para q_0 = MAD y F = {GUA} es el lenguaje vacío: L(A) = ∅.
- **Comportamiento del Sistema:**
  a. calcular_dijkstra extrae "MAD" de la cola de prioridad, verifica que no posee vecinos y finaliza el ciclo principal manteniendo distancias["GUA"] = float("inf"), retornando (None, [], []).
  b. El orquestador consultar_ruta en main.py detecta la ausencia de camino y lanza la excepción controlada ValueError("No existe una ruta de MAD hacia GUA.").
  c. El servidor HTTP responde con estado 400 Bad Request y carga útil JSON {"error": "No existe una ruta de MAD hacia GUA."}.

d. La interfaz web captura la respuesta, limpia las capas vectoriales activas del mapa, actualiza el sello de estado a "Sin ruta" y despliega la notificación del error al usuario.



5.5. Resumen Consolidado de Casos de Prueba

Caso	Origen (q_0) → Destino (F)	Cadena Generada (w_r)	Costo Total W(w_r)	Estado en el Autómata
Caso 1	GUA → MAD	GUA → SAL → MIA → MAD	8,950.00 km	Aceptada (w_r ∈ L(A))
Caso 2	GUA → JFK	GUA → SAL → MIA → JFK	3,610.00 km	Aceptada (w_r ∈ L(A))
Caso 3	PTY → MAD	PTY → MAD	8,200.00 km	Aceptada (w_r ∈ L(A))
Caso 4	MAD → GUA	Ninguna (L(A) = ∅)	∞ (Inalcanzable)	Rechazada / Sin Ruta

6. CONCLUSIONES

1. El modelado de la red aérea mediante un Autómata Finito Determinista Ponderado A = (Q, Σ, δ, q_0, F, w) permite formalizar rigurosamente el concepto de itinerario de vuelo como una cadena perteneciente a un lenguaje regular L(A), garantizando mediante la función reconocedora validar_ruta que ninguna ruta sugerida viole las restricciones direccionales de la topología.
2. La implementación del algoritmo de Dijkstra apoyada en montículos binarios (heapq) y diccionarios hash anidados resuelve el problema de optimización sobre el semianillo tropical en tiempo O((|V| + |E|) log |V|), demostrando que, aun cuando un camino posee mayor cantidad de transiciones (como GUA → SAL → MIA → MAD con 3 tramos y 8,950 km frente a GUA → PTY → MAD con 2 tramos y 9,550 km), el sistema identifica con exactitud el óptimo global en distancia.

3. La arquitectura libre de dependencias externas en el backend, combinada con la verificación cruzada entre el peso reportado por Dijkstra y la evaluación formal de la cadena en el autómata, dota al sistema AeroRoute de alta cohesión, trazabilidad matemática y tolerancia a fallos ante consultas hacia estados inalcanzables.